Enhancing Time Series Forecasting Using Synthetic Data

Evidence from Oil Price Prediction

JOSIAH LEE JIA XUAN
LOW JUN CHEN JASON
SINGARAVELAN DIVYASHREE
TIAN FENGYAO
MARIA JULIA SY
N ASHWIN KUMAR

DSE4101 Fintech Group 2
National University of Singapore

April 2026

1 Introduction

A central challenge in time series forecasting is the problem of data scarcity. Although the era of "Big Data" has transformed many areas of artificial intelligence since the early 2010s, time series applications often remain limited by relatively small effective sample sizes because they exhibit strong temporal dependencies that must be preserved. These dependencies restrict the generation and utilization of additional data, thereby reducing the amount of independent information available for model estimation.

Faced with these constraints, recent approaches have increasingly focused on extracting more information from existing data, often through the adoption of machine learning techniques that can better exploit complex patterns. This has led to a shift away from traditional econometric models toward more flexible frameworks capable of capturing nonlinear relationships and higher-order dependencies. More specifically, Giantsidi and Tarantola (2025) identify Long Short-Term Memory (LSTM) networks as a benchmark architecture within the class of Recurrent Neural Network (RNN) models. However, these models are typically data-intensive[1] and do not fully address the challenge posed by limited data availability in time series settings. Consequently, the development of robust and accurate time series models remains essential across a wide range of applications such as policymaking, financial risk management, and economic planning.

Motivated by this challenge, our study adopts a novel approach by focusing on the generation of synthetic data through machine learning techniques, with the aim of investigating whether such methods can improve the forecasting accuracy of time series models. Our approach seeks to alleviate the limitations imposed by data scarcity by enriching the effective sample size while preserving the underlying temporal structure and dependence of the data. If successful, this approach has the potential to enhance the robustness and predictive accuracy of time series models across a range of asset classes as well as in policy-oriented applications where reliable forecasts are critical for informed decision-making, while also providing a viable solution in settings where data availability is limited.

The empirical application of our study focuses on oil prices, given their central role in the global economy and their strong connections to macroeconomic conditions and financial markets. Therefore, our central research question is whether *data augmentation methods can enhance the forecasting accuracy of monthly crude oil prices*.

2 Data Description and Preparation

2.1 *Data*

Our study utilizes monthly West Texas Intermediate (WTI) crude oil prices as the target variable and focuses on a one-step-ahead forecasting horizon. Following Conlon et al. (2024), a set of macroeconomic predictors is incorporated to enrich the analysis. The list of predictors, along with brief descriptions, is provided below:

- **Inflation (INFL):** Growth rate of the U.S. Consumer Price Index (CPI);
- **Interest Rate (TBL):** Nominal rate on the U.S. 3-month Treasury bill;

[1] Empirical results from this study indicate that LSTM performance deteriorates as the training sample size decreases; see Figure 1 in the appendix.

- **Money Stock (M1 & M2):** Growth rates of U.S. M1 and M2 monetary aggregates;
- **Commodity Currencies:** Monthly nominal exchange rates, constructed as averages of daily observations, for four major commodity-exporting economies against the U.S. dollar: Australia (AUD), Canada (CAD), New Zealand (NZD), and South Africa (ZAR).

In total, nine explanatory variables are employed in this study, with data sourced from the Federal Reserve Economic Data (FRED) API and the FRED-MD database.

2.2 *Data Processing*

To ensure stationarity, a series of transformations is applied. For WTI prices and the four commodity currency exchange rates, logarithmic first differences are taken. Stationarity is subsequently verified using the Augmented Dickey–Fuller (ADF) test. The macroeconomic variables (INFL, TBL, M1, and M2) are transformed according to the procedures outlined in McCracken and Ng (2015) and are obtained from the FRED-MD database. After accounting for the loss of initial observations due to differencing, the final sample spans from February 1986 to January 2026, which reflects the earliest available period based on data availability. Due to missing observations for the Consumer Price Index (CPIAUCSL) from October 2025 to December 2025 at the time of data retrieval, these values are imputed using a forward-fill approach based on the most recent available observation from September 2025. Table 1 reports the summary statistics for all variables after transformation. The distributions and time-series plots are presented in the appendix (see Figures 2 and 4).

Table 1: Summary statistics for rate of growth of oil prices and predictor variables.

Variable	Mean	Std. Dev.	Skewness	Kurtosis	Minimum	Maximum	Autocorr.
wti_ret	0.0020	0.0960	-0.6485	7.4563	-0.5681	0.5456	0.2627
M2	-0.0000	0.0044	-0.2608	15.5876	-0.0326	0.0301	-0.1611
M1	0.0000	0.0749	-1.9299	227.3143	-1.1929	1.0942	-0.4717
TB3MS	-0.0073	0.1936	-1.1013	5.6439	-1.2300	0.7400	0.4990
CPI	-0.0000	0.0027	-0.3965	3.2545	-0.0126	0.0108	-0.1604
ZAR_USD_ret	0.0040	0.0342	0.7490	4.5175	-0.1228	0.1915	0.2931
NZD_USD_ret	0.0003	0.0257	-0.1728	0.6573	-0.1034	0.0740	0.2844
CAD_USD_ret	-0.0000	0.0159	0.5271	5.5366	-0.0601	0.1129	0.2693
AUD_USD_ret	-0.0001	0.0253	-0.8932	4.5181	-0.1732	0.0713	0.3409

The log returns of WTI display a near-zero mean and substantial volatility, coupled with negative skewness and pronounced excess kurtosis, suggesting a distribution characterised by downside risk and extreme observations. This is corroborated by the presence of pronounced spikes in the time-series plot, particularly during the COVID-19 pandemic, where unprecedented supply shocks and market disruptions led to abrupt and extreme price movements.

3 Time Series Data Augmentation Techniques

Time series data augmentation techniques aim to artificially generate additional useful observations to enhance model performance. Wen et al. (2020) classify these techniques into several distinct categories: (1) time-domain transformations; (2) frequency-domain transformations; (3) time–frequency

domain methods; (4) decomposition-based methods; and (5) learning-based methods. Each of these approaches is designed to manipulate the data while preserving its underlying temporal structure. Our study present three data augmentation approaches: *Jittering*, *Time-Series Generative Adversarial Networks (TimeGAN)*, and *Variational Autoencoder TimeGAN (VAE-TimeGAN)*. Jittering is used as a baseline because it introduces only minimal perturbations and is simple to implement. Next, we explore TimeGAN as documented by Yoon et al. (2019) due to its outstanding results in synthetic data generation. Yoon et al. find that TimeGAN is able to preserve temporal relationships and perform better than other GANs in similarity scores and predictive ability. We use GANs in order to generate similar but fundamentally distinct data to improve predictive performance However, given that TimeGAN is not tailored to our model, we propose a solution: a VAE-TimeGAN model to handle both cross-variable and temporal relationships.

3.1 *Jittering*

Jittering refers to the process of adding small amounts of Gaussian noise to observed data to perturb the series while preserving its underlying temporal and cross-sectional structure. It is a simple data augmentation technique for time series data commonly used for regularisation for deep learning models such as neural networks, where it helps improve generalisation by reducing overfitting. In this project, jittering acts as a baseline to compare against more advanced approaches; and is implemented by adding random noise to each series with variance set to 3% of the series' standard deviation, a calibration consistent with prior studies (Abbasi et al., 2024; Iwana and Uchida, 2021), which suggest that small-magnitude noise effectively enhances model robustness without distorting the underlying data-generating process.

3.2 *TimeGAN*

TimeGAN (Yoon et al., 2019) is a generative adversarial framework designed specifically for time series data. Unlike standard GANs that treat observations as independent draws, TimeGAN jointly models the marginal distribution of each variable, the cross-variable correlation structure, and the temporal dynamics of the sequence. It achieves this through five neural network components trained in three sequential stages.

3.2.1 Architecture

All five components are implemented as Gated Recurrent Unit (GRU) networks with sigmoid output activations. GRU is preferred over LSTM for this setting as it achieves comparable sequential modelling capacity with fewer parameters, reducing overfitting risk on the relatively small monthly dataset.

- **Embedder** (3-layer GRU, 128 units): maps real input sequences from the original feature space into a lower-dimensional latent representation, preserving temporal structure.
- **Recovery** (3-layer GRU, 128 units): the inverse mapping from latent space back to the original feature space. Together with the Embedder it forms a sequence autoencoder.
- **Generator** (3-layer GRU, 128 units): takes Gaussian noise as input and produces synthetic sequences in latent space, learning to match the distribution of real encoded sequences.

- **Supervisor** (2-layer GRU, 128 units): trained to predict the next latent state given the current one, directly supervising the Generator on step-by-step temporal transitions.
- **Discriminator** (3-layer GRU, 128 units): classifies sequences as real (from the Embedder) or synthetic (from the Generator), providing the adversarial training signal.

3.2.2 Training stages

Training proceeds in three stages. In Stage 1, the Embedder and Recovery are pre-trained as an autoencoder to reconstruct real sequences, establishing a stable and meaningful latent space before generation begins. In Stage 2, the Supervisor is trained on real latent sequences to minimise one-step-ahead prediction error, embedding temporal transition dynamics directly into the Generator. In Stage 3, all five components train jointly: the adversarial loss pushes the Generator to produce realistic sequences, while the supervised loss from Stage 2 is retained as a regulariser to prevent the Generator from sacrificing temporal coherence in favour of fooling the Discriminator.

3.2.3 Data and train–test split

The model is trained on nine monthly macroeconomic and commodity variables: WTI crude oil returns (`wti_ret`), four commodity-currency returns (AUD/USD, CAD/USD, NZD/USD, ZAR/USD), the Consumer Price Index (CPI), the 3-Month Treasury Bill rate (TB3MS), and broad money aggregates M1 and M2. The training period spans January 1987 to December 2015, comprising 348 monthly observations. A sliding window of length $T = 24$ months with stride 1 is applied to construct the training corpus, yielding $348 - 24 + 1 = 325$ overlapping sequences of shape 24×9. The out-of-sample evaluation period covers January 2019 to January 2026 (85 monthly observations), producing $85 - 24 + 1 = 62$ evaluation windows.

3.2.4 Sequence length selection

The sequence length $T = 24$ months is chosen on both statistical and domain grounds. From a domain perspective, a 24-month window spans two full annual cycles, capturing seasonal patterns in energy markets and the lagged transmission of monetary policy through CPI and money aggregates. A window shorter than 12 months would miss cross-annual dynamics; beyond 36 months, windows begin to overlap with distinct macroeconomic regimes (e.g. the 2008 financial crisis, the 2014 oil price collapse), making it harder for the model to learn a single coherent distribution rather than averaging across structurally different periods. From a statistical perspective, the available training span of 348 months constrains the trade-off between sequence richness and corpus size: $T = 24$ yields 325 training sequences, whereas $T = 36$ reduces this to 313 and $T = 48$ to 301. More critically, the learned distribution is statistically valid only within the trained window length; generation beyond T steps is architecturally possible (GRU hidden states can be unrolled arbitrarily) but distributional guarantees do not hold past step 24. Synthetic sequences are therefore generated as independent 24-month draws and stitched to form longer blocks, with the caveat that cross-window temporal continuity is not preserved.

3.2.5 Hyperparameters

The final model is trained with the configuration reported in Table 3.

Table 2: TimeGAN Hyperparameter Configuration.

Hyperparameter	**Value**
Sequence length (T)	24 months
Hidden dimension (all GRU layers)	128
Noise dimension	32
Batch size	64
Learning rate	5×10^{-4}
Embedding training steps	720
Supervisor training steps	720
Joint training steps (epochs)	1,200

3.3 *Empirical Results from TimeGAN*

When deciding which metrics are suitable to assess the TimeGAN and VAE-TimeGAN, we look at our goal which is to create synthetic data that is realistic, yet different from our existing data, or unseen. These 2 goals may appear conflicting, as on one hand, we want to generate realistic scenarios by getting the generative models to learn to produce something similar to our existing data, but on the other hand, we do not want it to produce the same data as our original as there would be no additional information learnt by the predictive models. Given this, we evaluated the quality of the TimeGAN-generated synthetic data in four complementary dimensions: marginal distribution fidelity, joint cross-feature structure, temporal dynamics, and overall discriminability. The first 3 metrics would test how similar our data is to the original distribution, while the last one aims to help assess our other goal.

3.3.1 Evaluation protocol

For each evaluation dimension, 62 synthetic windows of shape 24×9 are independently sampled from the trained model and compared against the 62 real out-of-sample windows from the 2019–2026 period. Because the synthetic windows are independent draws from the learned distribution and the real windows are drawn from a single contiguous series, the evaluation is distributional rather than sequence-to-sequence: the question is whether the synthetic population is statistically similar to the real population, not whether individual windows match.

Marginal distribution fidelity For each of the nine features, the real and synthetic observations are compared using two non-parametric statistics computed over the full set of observed values:

- **Kolmogorov–Smirnov (KS) statistic**: the maximum absolute difference between the empirical CDFs of the real and synthetic samples. A value of 0 indicates identical distributions; 1 indicates complete separation.
- **Wasserstein distance**: the minimum cost of transporting probability mass to transform the synthetic distribution into the real one, capturing differences in location, spread, and shape simultaneously.

Both statistics are averaged across all nine features to produce scalar summary measures.

Joint cross-feature structure The 9×9 Pearson correlation matrix is computed separately for the real and synthetic populations. The Frobenius norm of the matrix difference, $\|\mathbf{C}_{\text{real}} - \mathbf{C}_{\text{synth}}\|_F$, quantifies how well the synthetic data preserves the inter-variable relationships present in real data. A value of zero indicates perfect recovery of the joint correlation structure.

Temporal dynamics The autocorrelation function (ACF) of `wti_ret` is computed up to lag 12 on the raw contiguous 85-month evaluation series (not on windowed observations, to avoid inflating autocorrelation through overlapping repeated values) and on a synthetic series of equal length. The ℓ_2 distance between the two ACF vectors measures how well the model reproduces the temporal dependence structure of WTI returns.

Discriminative score A logistic regression classifier is trained on a balanced pool of 62 real and 62 synthetic windows (each flattened to a 216-dimensional vector), using a 70/30 train–test split. The ROC-AUC on the held-out set measures overall indistinguishability: an AUC of 0.5 indicates that synthetic sequences are statistically indistinguishable from real ones, while an AUC approaching 1.0 indicates that the synthetic data is easily detected as artificial. This is the most holistic metric, as the classifier may exploit any discriminating signal — marginal, joint, or temporal — and it operates at the window level without repetition artefacts.

3.3.2 Results

The evaluation results are reported in Table 3.

Table 3: TimeGAN out-of-sample evaluation results (January 2019 – January 2026).

Dimension	**Metric**	**Value**	**Target**
Marginal distribution	Avg. KS statistic	0.145	$\to 0$
	Avg. Wasserstein distance	0.020	$\to 0$
Joint structure	Correlation Frobenius norm	2.089	$\to 0$
Temporal dynamics	ACF ℓ_2 (wti_ret)	0.379	$\to 0$
Discriminability	Discriminative AUC	0.623	$\to 0.5$

The synthetic data achieves reasonable marginal fidelity: an average KS statistic of 0.145 and Wasserstein distance of 0.020 indicate that the per-feature value distributions are broadly similar to those observed in the real out-of-sample period. The temporal ACF distance of 0.379 on WTI returns suggests that the model reproduces some but not all short-horizon autocorrelation structure within the 24-month window.

The principal weakness is the correlation Frobenius norm of 2.089, which indicates that the synthetic data only partially preserves the cross-feature correlation structure of the real data. This is a known limitation of the standard TimeGAN formulation, which does not contain an explicit objective term penalising divergence in the joint cross-variable distribution; the adversarial and supervised losses operate primarily in the latent sequence space. The discriminative AUC of 0.623 (gap of 0.123 from

the ideal 0.5) is consistent with this finding: a linear classifier is able to distinguish synthetic from real windows at above-chance accuracy, reflecting the residual distributional discrepancy in the joint feature space. These limitations motivate the VAE-TimeGAN hybrid introduced in the following section.

3.4 *VAE-TimeGAN*

The standard TimeGAN evaluation reveals a structural limitation: the Frobenius norm of 2.089 on the correlation matrix indicates that cross-variable relationships are only partially preserved. This is an architectural consequence rather than a tuning failure. The adversarial and supervised objectives operate entirely within the embedding space and contain no explicit penalty for divergence in the joint cross-variable structure of the reconstructed output. The model is incentivised to produce temporally plausible sequences, but not necessarily sequences that preserve contemporaneous dependencies such as the co-movement between commodity currencies and WTI returns.

The VAE-TimeGAN hybrid addresses this by decomposing multivariate generation into two dedicated sub-problems. A VAE first compresses the nine-dimensional observation at each timestep into a structured latent code, explicitly capturing cross-variable relationships, TimeGAN then operates on these latent sequences, learning temporal dynamics in a space already regularised to respect cross-variable structure. The design draws conceptual inspiration from Yin et al. (2021), whose VAECGAN framework demonstrates that a regularised probabilistic bottleneck provides a smoother and more learnable target distribution for a downstream adversarial model. The theoretical motivation is threefold. First, operating in a compressed latent space reduces the effective dimensionality of the generative problem, which is consequential given the small training corpus of 312 monthly windows. Second, the VAE's KL divergence penalty enforces a structured prior, encouraging the encoder to produce a smooth distribution from which the TimeGAN generator can sample coherently. Third, the explicit decoupling of cross-variable and temporal learning reflects an inductive bias well-suited to macroeconomic data, where within-period covariance structure and across-period dynamics are governed by distinct economic mechanisms.

3.4.1 Architecture — Stage 1: Variational Autoencoder

The VAE follows the original formulation of Kingma and Welling (2013), which frames representation learning as approximate variational inference. The encoder $q_\phi(\mathbf{z} \mid \mathbf{x})$ and decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$ are trained jointly by maximising the Evidence Lower Bound (ELBO),

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q_\phi(\mathbf{z} \mid \mathbf{x}) \,\|\, \mathcal{N}(\mathbf{0}, \mathbf{I}))\,, \tag{1}$$

where the first term penalises reconstruction error and the second regularises the posterior towards the standard Gaussian prior, preventing degenerate deterministic encoding.

The VAE is implemented as a per-timestep MLP operating on each of the T=24 timesteps independently. At each timestep t, the encoder maps $\mathbf{x}_t \in \mathbb{R}^9$ through a shared linear-ReLU layer to a 64-dimensional hidden state, from which two separate linear heads produce the posterior mean $\boldsymbol{\mu}_t \in \mathbb{R}^4$ and log-variance $\log \boldsymbol{\sigma}_t^2 \in \mathbb{R}^4$. The log-variance is clamped to $[-4, 4]$ before exponentiation for numerical stability. A latent sample is drawn via,

$$\mathbf{z}_t = \boldsymbol{\mu}_t + \boldsymbol{\epsilon} \odot \exp\!\left(\tfrac{1}{2} \log \boldsymbol{\sigma}_t^2\right), \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \tag{2}$$

allowing gradients to propagate through the sampling operation. The decoder mirrors this structure, mapping $\mathbf{z}_t \in \mathbb{R}^4$ through a linear-ReLU layer back to $\mathbb{R}^9$ with no final activation, consistent with the unbounded reconstruction targets of the log-differenced, MinMax-scaled inputs.

3.4.2 Architecture — Stage 2: TimeGAN on Latent Sequences

The TimeGAN architecture used in Stage 2 has been described in full in Section 3.2 and is not repeated here. The sole structural modification relative to the standard formulation is that, at generation time, the VAE decoder described above replaces TimeGAN's native Recovery network. Because the TimeGAN in this hybrid operates on four-dimensional latent sequences rather than nine-dimensional raw data, its embedding space *is* the VAE latent space, and the VAE decoder directly performs the mapping from latent codes back to the original data space that the Recovery would otherwise handle. The end-to-end generation pipeline is consequently: sample Gaussian noise $\mathbf{z}_{1:T} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ of shape (T,4), pass through the TimeGAN Generator to obtain a raw latent sequence, refine through the Supervisor to enforce step-wise temporal transitions, and decode through the VAE decoder to recover a synthetic nine-dimensional normalised window of shape (24,9).

3.4.3 VAE-TimeGAN Training Procedure

Training proceeds in two sequential and independent stages, with the VAE trained to convergence before any TimeGAN training begins.

Stage 1: VAE Training The VAE is optimised using the Adam optimiser with a learning rate of 1×10^{-4} for 400 epochs with a batch size of 25. At each step, each training window of shape (24,9) is flattened along the temporal axis so that the encoder receives batch_size $\times$ 24 independent timestep observations per gradient update. The final 15% of windows are held out as a temporally ordered validation set.

The training objective is the ELBO, comprising a mean squared error reconstruction term and a weighted KL divergence with two modifications introduced to combat posterior collapse,

$$\mathcal{L}_{\text{VAE}} = \frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2 \;+\; \beta_t \cdot \sum_{k=1}^{4} \max\left(\lambda,\; \frac{1}{B}\sum_{b=1}^{B} \mathrm{KL}_k^{(b)}\right), \tag{3}$$

where $\mathrm{KL}_k^{(b)} = -\frac{1}{2}(1 + \log \sigma_k^{(b)2} - \mu_k^{(b)2} - \sigma_k^{(b)2})$ is the per-dimension KL, averaged over the batch before clamping. The free-bits threshold $\lambda = 0.1$ nats enforces a minimum information content of 0.4 nats across all four latent dimensions, preventing the encoder from collapsing to the prior. The annealing schedule,

$$\beta_t = \min\left(\beta_{\max},\; \beta_{\max} \cdot \frac{t}{t_{\text{warmup}}}\right), \quad \beta_{\max} = 0.005, \quad t_{\text{warmup}} = 50, \tag{4}$$

linearly increases the KL weight over the first 50 epochs so that the reconstruction objective dominates early training before regularisation pressure is introduced. The ceiling of $\beta_{\max} = 0.005$ is set well below the canonical $\beta = 1$, reflecting the difficulty of regularising a per-timestep encoder on a small, strongly autocorrelated dataset. Gradient norms are clipped to 1.0 throughout. Full hyperparameters are reported in Table 2.

Stage 2: TimeGAN Training Likewise with the architecture description, the training process of our TimeGAN models has also been explained previously. The main difference is that it is now trained on the latent space instead of the original variable space. The final hyperparameters are available in Table 3 found in the appendix.

3.4.4 VAE Tuning and Failure Analysis

The principal challenge during VAE training was posterior collapse, in which the encoder drives all posterior variances towards zero, the KL term in the ELBO approaches zero, and the latent space loses its generative properties. Three successive interventions were required before an acceptable posterior was obtained, summarised in Table 1. The initial configuration of eight latent dimensions with $\beta = 1.0$ produced a KL at convergence of approximately 0.0001 nats and posterior standard deviations of 0.004–0.009, indicative of complete collapse. Reducing β to 0.01 overcorrected, raising the KL to 0.560 but with regularisation still dominating the small reconstruction signal. Introducing the free bits heuristic partially improved matters, yielding posterior standard deviations of 0.05–0.18, but the decisive intervention was reducing the latent dimensionality from eight to four. The tighter bottleneck forced the encoder to utilise each dimension rather than distributing information redundantly across partially-used channels. The final configuration produced a KL of 0.400 and posterior standard deviations of 0.38–0.94, the first configuration in which the encoder demonstrably engaged the stochastic bottleneck, though residual under-utilisation relative to the unit Gaussian ideal remained.

The loss curves in Figure 6 corroborate this account. The total ELBO converges smoothly by epoch 100 with negligible train–validation gap. The KL loss begins at the free bits floor of 0.40 nats, peaks at approximately 0.688 nats around epoch 80 as the encoder responds to increasing KL pressure, then retreats to the 0.40 nats floor, characteristic of a model that partially resists regularisation before the small dataset and per-timestep architecture prevent it from sustaining a higher KL. The reconstruction loss reaches near zero by epoch 50 and remains stable thereafter, with train and validation curves tightly overlapping throughout.

3.4.5 TimeGAN Tuning and Failure Analysis

The dominant failure mode in the TimeGAN component was discriminator dominance, visible in Figure 7. Phases 1 and 2 proceeded as expected: the embedding loss descended from approximately 1.1 to near zero within 75 epochs and the supervised loss remained low throughout, though convergence was slower than the standard TimeGAN, consistent with the four-dimensional latent inputs presenting a more compressed signal requiring more careful fitting. In Phase 3, the Generator and Discriminator began in approximate balance at approximately 2.0, but at around epoch 300 the adversarial equilibrium collapsed irreversibly: the Discriminator loss fell to near zero whilst the Generator loss climbed monotonically from 2.0 to over 12, indicating that the Discriminator had achieved near-perfect separation and the gradient signal to the Generator became uninformative. The discriminative score corroborates this: starting at 0.73 at epoch 50, it reached its best value of 0.6125 at epoch 150 before deteriorating to a volatile range of 0.70–0.85, never approaching the ideal of 0.5. The epoch 150 checkpoint is selected by early stopping for all subsequent generation.

The discriminative score trajectory (right panel) provides complementary evidence. The score begins at approximately 0.73 at the first evaluation checkpoint (epoch 50), dips to its best value of 0.6125

at epoch 150, and then deteriorates irregularly, reaching values as high as 0.90 before settling in a volatile range of approximately 0.70–0.85 for the remainder of training. The best checkpoint at epoch 150 is selected by early stopping and used for all subsequent generation. Notably, the score never approaches the ideal value of 0.5, confirming that even at the best checkpoint the synthetic sequences remain distinguishable from real ones, though substantially less so than at later epochs.

Several countermeasures were applied in attempting to stabilise joint training. A threshold-based discriminator gating condition was first introduced, withholding discriminator gradient updates when its loss fell below a specified threshold; however, because the discriminator loss in this implementation is the sum of three binary cross-entropy terms (one for real sequences, two for different synthetic sequences), the aggregate loss rarely fell below the threshold, rendering the gating condition effectively inactive. This was replaced by the epoch-alternating update rule described in the training procedure section, which gave the Generator twice as many gradient updates per epoch as the Discriminator and proved more effective in delaying collapse. The moment matching coefficients were reduced from 100 to 5 after the original values were found to overwhelm the adversarial signal entirely, producing sequences whose moments matched the real data but whose temporal dynamics were degenerate. Despite these interventions, the collapse at epoch 300 persisted across all configurations.

3.5 *Empirical Results from VAE-TimeGAN*

3.5.1 Evaluation protocol

The VAE-TimeGAN hybrid is evaluated on the 312 normalised training windows using the same metric framework as the standard TimeGAN: marginal distribution fidelity (KS statistic), joint cross-feature structure (correlation Frobenius norm), temporal dynamics (mean ACF MAE across all nine variables), distributional divergence (MMD), and overall discriminability (discriminative accuracy). Results are reported in Table 4 alongside standard TimeGAN figures where a valid comparison exists. Two contextual differences should be noted: the VAE-TimeGAN is evaluated on training windows whilst the standard TimeGAN results in Section 3.2 use 62 held-out out-of-sample windows from 2019–2026, and the reported discriminative accuracy of 1.000 reflects end-of-training evaluation, whereas the best-checkpoint score of 0.6125 at joint epoch 150 better characterises the model at the point of deployment.

3.5.2 Results of VAE-TimeGAN and comparison with standard TimeGAN.

The dominant finding is severe mode collapse. Across all nine variables, synthetic standard deviations are a small fraction of their real counterparts: std ratios range from 0.001 (M2) to 0.158 (CPI), with a median of approximately 0.05, indicating that the synthetic sequences cluster tightly around the per-variable mean with virtually no dispersion. The PCA and t-SNE visualisations in Figure 5 make this explicit: the synthetic windows form a compact cluster at the centre of the real data distribution, consistent with mean-collapsed generation. All remaining metrics are downstream consequences of this failure. The KS statistic of 0.543 reflects near-complete separation between empirical CDFs, substantially worse than the standard TimeGAN's 0.145. The Frobenius norm of 5.847 is nearly three times the standard TimeGAN's 2.089, indicating that near-constant synthetic sequences produce a degenerate correlation matrix, the opposite of the cross-variable fidelity the VAE stage was designed to provide. The discriminative accuracy of 1.000 confirms perfect separability across all three evaluation splits.

The single exception is temporal fidelity. The mean ACF MAE of 0.177 across nine variables compares favourably to the hybrid configuration (0.203) evaluated on the same windows, and the per-variable ACF plots in Figure 5 show that the synthetic ACF tracks the real decay profile reasonably well for smoother series, most notably M2 (MAE=0.132), CAD_USD (MAE=0.101), and TB3MS (MAE=0.157). Variables with irregular or oscillatory ACF profiles, including WTI (MAE=0.236) and ZAR_USD (MAE=0.216), are reproduced with lower fidelity. That temporal structure survives whilst distributional coverage collapses suggests the TimeGAN Supervisor successfully embedded autocorrelation patterns into the latent sequences even as the Generator failed to produce meaningful variance.

In aggregate, the VAE-TimeGAN does not achieve comparable synthetic data quality to standard TimeGAN. The VAE's explicit cross-variable compression stage was entirely negated by downstream generator collapse, yielding worse performance on every distributional metric despite the added architectural complexity. The results constitute a negative empirical finding: the hybrid's theoretical motivation is sound, but its realisation is contingent on resolving the posterior collapse and prior-hole problems before the architectural decomposition can deliver its intended benefits. The diagnosis of these failure modes, and the directions they point towards, recurrent VAE encoders, VQ-VAE priors, and conditional generation, are discussed in the limitations and future work section.

4 Forecasting Methodology

4.1 *Forecasting models for WTI returns*

Ly et al. (2021) demonstrate that LSTM models can outperform traditional approaches, such as ARIMA, in forecasting commodity prices. Consequently, the LSTM architecture is selected as the primary model in this study, given both its strong empirical performance and its widespread adoption as a benchmark model in deep learning for sequence modelling tasks. To maintain model parsimony, and in light of the limited effective sample size, a relatively shallow LSTM architecture is adopted, comprising two LSTM layers, two dropout layers, and a single dense output layer. A simplified grid search, following Ly et al. (2021), is adopted to tune the model hyperparameters (lookback window, dropout rate, number of units, and number of epochs). Lastly, Autoregressive (AR) and autoregressive distributed lag (ADL) models are employed as benchmark models for comparison.

To evaluate model performance across varying levels of complexity, the forecasting models are estimated under three feature specifications. First, a univariate specification using only WTI returns is considered, serving as a baseline benchmark. Second, a bivariate specification incorporating WTI returns and the 3-month Treasury Bill rate (TB3M) is employed to introduce additional macroeconomic information. Finally, a multivariate specification including all eight additional covariates described in Section 2 is implemented. This sequential specification strategy enables a systematic evaluation of how forecasting performance evolves as the dimensionality of the input space increases. In particular, it allows for an assessment of whether the effectiveness of data augmentation techniques varies across models of differing complexity.

4.2 *Sample Splitting and Model Evaluation*

Our full original sample consists of 480 observations. A 70/15/15 split is adopted, yielding 336 observations for training, 72 observations for hyperparameter tuning, and 72 observations for out-of-sample testing. In addition, 336 synthetic observations are generated using the three data augmentation techniques described in Section 3 and are appended prior to the original training data, effectively doubling the size of the training set only for the machine learning model (LSTM) used in forecasting.

More importantly, the validation and out-of-sample test sets consist exclusively of original (non-augmented) observations and are never exposed to synthetic data at any stage, ensuring a fair and unbiased evaluation of model performance.

Finally, an expanding window framework is employed, whereby the models are re-estimated at each time step using all available information up to that point. At each step, the input variables are re-scaled using only historical data, ensuring consistency with real-time prediction and preventing the incorporation of future information, thereby avoiding look-ahead bias. This approach also allows the model to fully exploit the increased data availability afforded by the augmented training set. The optimal hyperparameters used for model estimation are reported in Table 5 and Table 6, where the configurations are selected based on validation set performance.

5 Empirical Results

Having established the properties of the synthetic data generation methods in Section 3, we now evaluate whether these augmentation techniques improve downstream forecasting and trading performance. Overall, the results indicate that the usefulness of synthetic data is highly context dependent. While some gains are observed in simpler specifications, particularly in the univariate WTI setting, these improvements do not generalise consistently once additional macroeconomic covariates are introduced.

5.1 Forecast Accuracy

Table 7 reports forecast performance for the benchmark models and the LSTM variants under the three feature specifications: WTI only, WTI with TB3M, and WTI with all covariates. Across all three settings, the benchmark econometric models outperform the LSTM-based models on RMSE and MAE. This suggests that, in a monthly forecasting environment with limited effective sample size, the simpler econometric benchmarks remain more robust than the more flexible neural network alternatives.

Among the LSTM variants, the clearest gains from augmentation arise in the WTI-only specification. In the full sample, TimeGAN records the lowest RMSE and MAE among the augmented LSTM models, indicating that synthetic data may be useful when the forecasting problem is relatively simple and focused solely on the target series. However, this advantage does not persist in the richer feature specifications. In both the TB3M and all-covariates settings, augmentation does not yield systematic improvements over the baseline LSTM, and in several cases the augmented models perform materially worse. This pattern suggests that the marginal value of synthetic data diminishes as the dimensionality of the input space increases.

As a robustness check, we repeat the forecast evaluation after excluding the COVID-19 crash period. The corresponding results are reported in Table 8. As expected, forecast errors decline across models

once this shock is removed. However, the main ranking remains consistent, with the benchmark models continuing to dominate across all three specifications, and the augmented LSTM variants continue to show their strongest relative performance only in the WTI-only case. In particular, all synthetic-data LSTM variants outperform the baseline LSTM in the ex-COVID univariate setting, but augmentation still fails to produce consistent gains in the TB3M and all-covariates cases.

5.2 Trading Performance

To evaluate whether the forecasts contain useful trading information, we map the monthly model predictions into directional trading positions and compare the resulting strategies against a buy-and-hold benchmark. Strategy performance is assessed using annualized Sharpe ratio, annualized return, cumulative return, and maximum drawdown. The full sample trading results are reported in Table 9 and illustrated in Figures 12, 13, and 14.

In contrast to the forecast accuracy results, the trading results are more favourable to the LSTM-based strategies. Across the three feature specifications, the active trading rules materially outperform buy-and-hold. However, the best-performing model differs across settings. In the WTI-only specification, TimeGAN delivers the strongest trading performance. In the TB3M specification, jitter achieves the highest Sharpe and cumulative return. In the all-covariates specification, the baseline LSTM outperforms the augmented variants and records the strongest overall trading results. These findings indicate that economic usefulness is highly sensitive to the specification of the forecasting problem and that no single augmentation method dominates across all environments.

We further rerun the backtest after excluding the COVID crash window. The ex-COVID trading results are reported in Table 10. Once the extreme crash period is removed, the results become more stable and easier to interpret, but the overall ranking remains consistent. This supports the conclusion that augmentation is most useful in the simpler forecasting settings and does not provide uniform improvements once the model incorporates a broader set of covariates.

A particularly notable result emerges in the WTI-only case. This specification exhibits the clearest benefit from augmentation, with all synthetic methods improving on the baseline LSTM and TimeGAN generating the strongest overall trading return. This suggests that synthetic augmentation may be most useful when the model is focused directly on the dynamics of the target series, rather than attempting to jointly model a larger and potentially noisier predictor set.

5.3 Discussion

Taken together, the empirical results point to three main conclusions. First, benchmark econometric models remain stronger on conventional forecast accuracy metrics. This indicates that in a low frequency monthly setting with limited effective sample size, simple benchmark models remain difficult to outperform in point forecasting.

Second, the benefits of synthetic data augmentation are highly specification dependent. Standard TimeGAN shows the most promise in the univariate WTI-only setting, where it improves both forecast-related and trading-related outcomes relative to the baseline LSTM. However, these gains do not generalise consistently to richer feature spaces. As more macroeconomic signals are introduced, the marginal value of synthetic data weakens and, in some cases, disappears altogether.

Third, the contrast between forecast accuracy and trading performance is itself informative. Even where the LSTM variants underperform benchmarks on RMSE and MAE, some augmented models still generate economically useful directional signals, particularly in the simpler specifications. This suggests that the value of synthetic data may lie less in improving precise point forecasts and more in enhancing the practical usefulness of the signals under certain conditions.

Overall, the evidence suggests that synthetic data augmentation can be beneficial, but only under specific conditions. Its effectiveness depends not only on the quality of the generated data, but also on the complexity of the feature space and the objective of the forecasting exercise. In our application, synthetic augmentation appears most effective when the task is relatively simple and the model focuses directly on the target dynamics, but much less reliable once the forecasting environment becomes high-dimensional and multivariate.

6 Contributions, Limitations, and Future Directions

Our project represents a novel contribution by developing one of the first rigorous comparative frameworks for assessing the effectiveness of data augmentation in time-series forecasting, specifically in the context of oil price forecasting. Our results suggest that although deep learning models such as LSTMs are often constrained by limited data availability, TimeGAN-based synthetic data generation shows promising potential in enhancing forecasting performance in the univariate setting. However, these gains do not generalise uniformly across model specifications. As more economic signals are introduced, the incremental value of synthetic data diminishes where there is little or no improvement observed in bivariate and higher-dimensional settings. This highlights the need for caution when generalising the benefits of data augmentation and underscores that its effectiveness is highly context-dependent

We identified several limitations in our project. Firstly, our naive method of row-wise concatenation of the synthetic to real data may not fully respect the underlying temporal continuity of the time-series data, potentially introducing distortions at the transition point. Secondly, the dataset utilised in this study is relatively limited in size. Although data augmentation was intended to alleviate data scarcity, the TimeGAN framework is inherently data-intensive and requires a sufficiently large sample to learn meaningful temporal dynamics. As a result, the limited dataset may have constrained the training of the generative model, leading to lower-quality synthetic data and, in turn, adversely affecting the performance of the LSTM forecasting model Thirdly, our macroeconomic covariates were selected based on our literature review rather than more formal and robust feature selection methods, which may have introduced noise or reduced model efficiency in the multivariate setting. Finally, the experimental results are based on a single random seed. Given the stochastic nature of neural network training, this may introduce variability in model performance and limit the robustness of the findings. A more rigorous approach would be to evaluate the models across multiple random seeds and report the average performance to provide a more reliable and stable assessment.

Building upon the results of this study, several key areas can be prioritised for future research. One such improvement could be the development of sequence-aware augmentation methods that preserve temporal patterns and dependencies more effectively, moving away from the simplified process of row-wise concatenation. Future studies could also leverage higher-frequency data or larger datasets to enhance the training process of GAN-based frameworks to improve overall performance. Furthermore,

exploring more complex neural architectures, such as Recurrent VAEs or attention-based Transformer models, could significantly refine how latent information is extracted and compressed from macroeconomic series. Additionally, the use of feature selection methods like Principle Component Analysis (PCA) would provide more evidence based reasoning for determining which covariates should be included in the model to improve the precision and reliability of data augmentation. Lastly, future research could examine the applicability of data augmentation techniques within traditional econometric frameworks, thereby assessing whether similar performance gains extend beyond deep learning models. It would also be valuable to systematically investigate the relationship between the extent of data augmentation and forecasting performance, with the aim of identifying the level of augmentation required to achieve stable and reliable predictions.

7 Detailed Code and Data Section

Our code is available here https://github.com/Meowth239/dse4101-project.

7.1 Data

The data sample used in this study was collected and processed as follows. Daily observations of four commodity currency exchange rates were retrieved from the Federal Reserve Economic Data (FRED) database via the FRED API and resampled to a monthly frequency by averaging daily values, as implemented in the `predictor_data_extraction.ipynb` script. Monthly West Texas Intermediate (WTI) crude oil prices were similarly obtained from FRED, covering the period from January 1986 to January 2026, which defines the sample endpoint. The remaining four macroeconomic variables were sourced from the FRED-MD database using the `FredMD.r` script and pre-transformed for stationarity according to the transformation codes in Ng and McCracken (2015). Subsequently, the `covariate_explore.ipynb` notebook was used to conduct formal stationarity testing using the Augmented Dickey–Fuller (ADF) test and to apply any additional necessary transformations. Once stationarity was achieved across all series, the variables were consolidated into a single dataset and exported as a final CSV file (`final_data.csv`), containing the date index and all processed features. In addition, augmented datasets were constructed by appending synthetic observations generated using TimeGAN and VAE-TimeGAN. These are stored as `timegan_stitched_with_real.csv` and `final_data_with_VAETIMEGAN.csv`, respectively.

7.2 TimeGAN

The TimeGAN implementation is organised across two execution notebooks and a set of helper functions defined within the primary notebook, with a clear separation between model training and evaluation on one side, and downstream data generation on the other.

The main notebook, `timeGAN_multivariate_analysis.ipynb`, handles the full pipeline from data ingestion to artifact export. It is structured around seven helper functions.
`load_and_validate_data` reads `final_data.csv`, parses the date column, sorts chronologically, and validates the dataset for missing values, duplicate dates, and missing months, returning a profile summary alongside the cleaned dataframe and feature column list. `select_period` slices

the dataframe to a specified date range, used to isolate the training period (January 1990 – December 2018) and the evaluation period (January 2019 – January 2026) with no overlap. `make_windows` constructs overlapping sliding windows of shape `(num_windows, T, 9)` from a contiguous array using a stride of 1, producing 325 training windows and 62 evaluation windows at sequence length $T = 24$. `train_timegan` fits a per-variable `MinMaxScaler` to the training period, scales the data to $[0, 1]$, and trains a `TimeSeriesSynthesizer` from the `ydata-synthetic` library using the GRU-based TimeGAN configuration; the fitted scaler is returned alongside the trained synthesiser for consistent inversion at generation time. `sample_synthetic` calls the trained synthesiser to draw n independent windows of shape `(n, T, 9)` in scaled space and applies the inverse transform to recover original-scale values. `eval_metrics` accepts real and synthetic window arrays and computes the full evaluation suite: KS statistic and Wasserstein distance per feature (averaged across nine features), the Frobenius norm of the correlation matrix difference, the ACF ℓ_2 distance on `wti_ret` computed on the raw contiguous evaluation series to avoid repeated-observation bias, and the discriminative AUC from a logistic regression classifier trained to distinguish real from synthetic windows. `apply_timegan_speed_patches` monkey-patches the `ydata-synthetic` training loop to enable stage-specific step ratios and to replace the default Adam optimiser with the legacy Keras implementation required for Apple Silicon compatibility, without altering the mathematical training procedure.

Following training, the notebook exports three artifacts to `data/timegan_outputs/`: the serialised synthesiser (`timegan_model.pkl`), the fitted scaler (`timegan_scaler.pkl`), and a JSON metadata file (`timegan_artifact_metadata.json`) recording the feature column list, training period boundaries, sequence length, and all hyperparameters. These artifacts decouple training from generation, ensuring that the downstream notebook can reproduce identical outputs without re-running the computationally expensive training procedure.

The second notebook, `timeGAN_generate_downstream.ipynb`, is responsible solely for synthetic data generation and export. It loads the three artifacts, resolves the desired stitched block length from user-facing controls (`stitch_total_months` or `stitch_num_windows`), and produces two output formats. First, n independent 24-month windows are generated and written to `timegan_synthetic_sequences_long.csv` in long format with `sample_id` and `step` columns, suitable for window-level analysis. Second, a larger batch of windows is generated and reshaped into a contiguous synthetic block of shape `(num_windows × 24, 9)`, assigned synthetic monthly dates ending one month before the real data begins, and exported as `timegan_stitched_synthetic_only.csv`; when `export_stitched_with_real` is enabled, the synthetic block and the real history are concatenated and sorted by date, producing `timegan_stitched_with_real.csv`, the combined dataset used in the downstream forecasting experiments. Separating generation into a dedicated notebook allows the synthetic dataset to be regenerated with different volume settings at any time without re-executing the training pipeline.

7.3 VAE-TimeGAN

The VAE-TimeGAN pipeline is implemented across four reusable Python modules. `data_pipeline.py` handles all data preparation: it validates the input, fits a per-variable MinMax scaler to the training period, constructs overlapping 24-month sliding windows of shape (num_windows, 24, 9), and persists the fitted scaler to `scaler.pkl` for consistent inversion at generation time.

vae_module.py implements the VAE, exposing train_vae, encode, and decode functions; encode maps real windows to latent sequences of shape (num_windows, 24, 4) using the posterior mean for determinism, and decode maps latent sequences back to normalised data space. timegan_module.py implements the custom PyTorch TimeGAN, designed to accept either raw normalised windows or VAE latent sequences interchangeably, making it usable for both the standard and hybrid pipelines without modification; train_timegan runs all three training phases with early stopping, and generate produces synthetic windows at inference time. evaluation_module.py provides the evaluation suite via a single evaluate_all entry point, which computes all five metrics, produces the multi-panel figure, and writes both a text and JSON report; the module accepts pre-generated numpy arrays directly, ensuring identical evaluation code is applied across all three augmentation methods in the ablation study.

The three notebooks orchestrate the experiments. test_custom.ipynb runs the full VAE-TimeGAN pipeline end-to-end using the custom modules: it loads and preprocesses the data via data_pipeline, trains the VAE via vae_module, encodes the training windows into latent sequences, trains TimeGAN on those latent sequences via timegan_module, and calls evaluation_module to assess the output. test_ydata.ipynb documents the experiment with the ydata-synthetic library as an alternative TimeGAN implementation; it uses data_pipeline and vae_module for preprocessing and encoding but substitutes the ydata-synthetic TimeGAN for the custom implementation, and is retained as a record of a negative finding. gen_data.ipynb handles final synthetic data generation and visualisation: it loads the saved VAE checkpoint and the best TimeGAN checkpoint, generates the synthetic windows used in the downstream forecasting experiments, applies inverse_transform to recover the original scale, and produces the time series plots reported in the appendix. For the purpose of this project, we focus on test_ydata.ipynb as our main training script and gen_data.ipynb for generation.

7.4 LSTM

The core LSTM implementation is defined in my_lstm.py, which contains the primary functions for model construction and the expanding window training framework. In particular, The function build_lstm_model specifies a two-layer LSTM architecture with two dropout layers, trained using the Adam optimiser with a learning rate of 0.001 and mean squared error (MSE) loss. The function expanding_window_lstm_forecast implements the core expanding window framework, where the model is re-trained at each time step using all available historical data to generate one-step-ahead forecasts. The function expanding_window_lstm_forecast3 extends this framework by incorporating jittering-based data augmentation. The accompanying notebooks (e.g., lstm-*.ipynb) serve as experiment drivers, each specifying different configurations such as model variants and data augmentation settings, while reusing the underlying implementation in my_lstm.py. All components are maintained within the lstm_src branch of the repository.

7.5 Forecast and Trading Analysis

The out-of-sample evaluation of forecasting accuracy and trading performance is implemented in a dedicated notebook, forecast_eval.ipynb. This notebook reproduces all forecast-accuracy and trading-performance results reported in the empirical-results section. It evaluates four LSTM variants

(baseline, jitter, TimeGAN, and VAE-TimeGAN) across three feature specifications: WTI only, WTI with TB3M, and WTI with all covariates. For forecast-accuracy comparison, it also loads benchmark model outputs for AR(3), ADL(1,1), and Full ADL from separately generated CSV files stored in the `benchmark/results/` directory.

The notebook is organised into a sequence of reusable helper functions and evaluation blocks. The setup section imports the required libraries, defines the shared configuration, and specifies the COVID exclusion window used in the robustness checks. A set of helper functions then handles loading result files, renaming prediction columns, merging outputs across LSTM variants, plotting forecast comparisons, computing forecast-accuracy metrics, excluding the COVID crash period, and formatting summary tables. This structure ensures that the same evaluation logic is applied consistently across all three feature specifications.

Three merged evaluation dataframes are constructed for the LSTM variants. Each dataframe contains the realised out-of-sample return (`actual`) together with the four LSTM forecasts (`normal`, `jitter`, `timegan`, and `vaetimegan`). These correspond respectively to the WTI-only, WTI + TB3M, and all-covariates specifications. The benchmark outputs are loaded separately from `ar_test_results.csv`, `adl-1_results.csv`, and `adl-8_results.csv`. These benchmark rows are appended only at the forecast-evaluation stage, with AR(3) assigned to the WTI-only specification, ADL(1,1) to the WTI + TB3M specification, and Full ADL to the all-covariates specification.

Forecast accuracy is evaluated using RMSE, MAE, and Directional Accuracy (DA). For the LSTM models, these metrics are computed from the merged wide-format evaluation dataframes. The notebook then repeats the same procedure after excluding the COVID-19 crash window from February 2020 to June 2020, generating the ex-COVID forecast-accuracy results reported in the robustness analysis.

The notebook also implements the trading-performance analysis for the LSTM variants. A set of trading helper functions converts the monthly forecasts into directional positions using a median-absolute-signal threshold rule, applies a turnover-based transaction cost, and computes monthly net returns. A `run_trading_simulation` wrapper then evaluates each feature specification against a buy-and-hold benchmark and reports annualized Sharpe ratio, annualized return, cumulative return, and maximum drawdown. For each specification, the notebook also generates cumulative-return and monthly-return plots.

Finally, the same trading simulation is rerun after excluding the COVID crash window, producing the ex-COVID trading summary tables and plots for the WTI-only, TB3M, and all-covariates specifications.

8 Reflection

We started this study out of a growing fatigue with repeatedly applying a wide range of models—spanning linear, nonlinear, machine learning, and deep learning approaches—that we had encountered throughout our four years of study. Across these time series forecasting exercises, a recurring theme emerged: the persistent challenge of limited data and the adverse effects it can have if not properly addressed. Although we learned the methodological tools to confront such issues, this observation prompted us to shift our focus toward a deeper investigation of data scarcity as a fundamental constraint in time series forecasting. This led us to explore whether machine learning techniques could be leveraged to generate synthetic data as a targeted approach to addressing this issue. Our initial empirical application consid-

ered a broad set of commodity assets. However, this scope proved too expansive for a focused analysis and, therefore, we narrowed our study to crude oil prices to allow for a more in-depth and coherent investigation.

Following the success of our Phase 1 findings, which focused on applying TimeGAN as the primary data augmentation method to WTI returns, our research was subsequently extended in several directions. First, we incorporated additional covariates that have been shown in the literature to be informative in predicting WTI returns. Specifically, we prioritised the 3 month treasury rate (TB3SM) as the main covariate in our bicovariate specification because Arora and Tanner (2012) demonstrated that this is a more significant driver of oil prices compared to other macroeconomic variables like exchange rate. Second, we incorporated other data augmentation techniques such as Jittering and our novel approach of VAE-TimeGAN to address some of the limitation from vanilla TimeGAN approach. We have also planned for a more complex, large-scaled project including copula modeling to understand the dependence structure between the variables. However, we decided to omit that for the purpose of narrowing down the scope. Third, we added econometric benchmark models to our analysis to provide a basis for comparison with the machine learning approaches.

While these enhancements broaden the scope of the analysis, they also introduce several methodological and practical challenges. In particular, the application of the standard TimeGAN framework revealed both technical and conceptual difficulties that informed subsequent design decisions. On the technical side, the `ydata-synthetic` implementation was incompatible with Apple Silicon out of the box, producing silent training failures caused by the default Adam optimiser conflicting with the Metal backend. We resolved this by monkey-patching the training loop to substitute the legacy Keras Adam implementation, and introduced stage-specific step ratios to reduce wall-clock training time without altering the mathematical procedure. On the modelling side, the evaluation results exposed a clear structural weakness: while marginal distribution fidelity was reasonable (average KS of 0.145, Wasserstein of 0.020), the correlation Frobenius norm of 2.089 indicated that the cross-variable joint structure was only partially recovered. We attributed this to the fact that the standard TimeGAN formulation contains no explicit objective term penalising divergence in the cross-feature correlation structure; the adversarial and supervised losses operate in a shared latent sequence space but do not directly enforce that the joint distribution of all nine variables is preserved. The discriminative AUC of 0.623 was consistent with this interpretation, suggesting that a linear classifier could exploit residual joint distributional discrepancies to distinguish synthetic from real windows. We also identified a fundamental dataset size constraint: with only 348 monthly training observations, the model had access to 325 overlapping windows, significantly fewer than the sample regimes for which TimeGAN was originally designed, limiting the Generator's ability to learn complex multivariate dependencies. These findings collectively motivated the VAE-TimeGAN hybrid: by first compressing the nine-variable input into a structured latent space via the VAE, we hypothesised that the TimeGAN would operate on a lower-dimensional representation where cross-variable relationships are already encoded, relieving the Generator of the burden of learning joint structure from raw sequences directly.

We also faced some technical hurdles along the way involving the VAE-TimeGAN approach, which required us to come up with tailored solutions to solve the them. The first issue we had to deal with was the posterior collapse of the VAE. We observed that our initial encoder ignored the stochastic bottleneck, outputting near-zero variance latent vectors and behaving like a plain autoencoder. We hypothesised that

the high autocorrelation in monthly macroeconomic data made the reconstruction task too easy for the encoder to exploit deterministically. To address this, we reduced latent dimensionality from 8 to 4. This forced genuine compression and higher posterior standard deviations, confirming the encoder was finally utilizing the latent space. Secondly, there was an issue of discriminator dominance in the TimeGAN. Our TimeGAN training was plagued by an overly powerful discriminator, leading to a sharp rise in generator loss past epoch 300. To prevent this collapse, we introduced epoch-alternating updates, giving the generator twice as many gradient steps to keep pace. While this improved our discriminative score, we learned that the fundamental constraint was our small dataset size, which was significantly smaller than the dataset size that TimeGAN is designed for.

On the data side, missing observations in the FRED-MD dataset were addressed using a forward-fill imputation approach to preserve continuity in the time series. From a computational perspective, the implementation of our models proved to be resource-intensive, necessitating the use of cloud-based computing infrastructure. This experience provided practical insight into the substantial computational demands of modern machine learning workflows and helps to explain the significant investments firms make in computing resources today

Against this backdrop, the evaluation of the results allowed us to understand the impact of data augmentation methods on model performance and trading strategy performance. We discovered that the success of augmentation is highly sensitive to the feature set. TimeGAN led in the WTI-only case, and Jitter performed best in the bivariate setting, but the baseline LSTM dominated in the all-covariate setup. This suggests that the incremental value of synthetic data decreases as we incorporate more covariates into the model and is highly specification-dependent.

While we do not fully resolve the data scarcity problem we initially set out to address, our findings highlight the considerable potential of synthetic data methodologies. In particular, they point to a promising and underexplored avenue for improving time series forecasting, with meaningful implications for both academic research and practical applications. More importantly, our results suggest that improvements in forecasting performance need not be driven solely by increasingly complex models, but can instead be achieved through more effective use and enhancement of the underlying data. More broadly, our study highlights the importance of systematic experimentation, as evaluating a range of approaches allows us to uncover not only the strengths of these methods but also their limitations. Ultimately, progress in research is inherently incremental, and gaining insight into the conditions under which data augmentation fails may be just as valuable as understanding when it succeeds.

Lastly, we would like to express our sincere gratitude to Dr Ta-Cheng Huang and our fellow coursemates for their guidance and support throughout this project. Their valuable insights and feedback played a crucial role in shaping the direction and outcomes of this study.

A Appendix

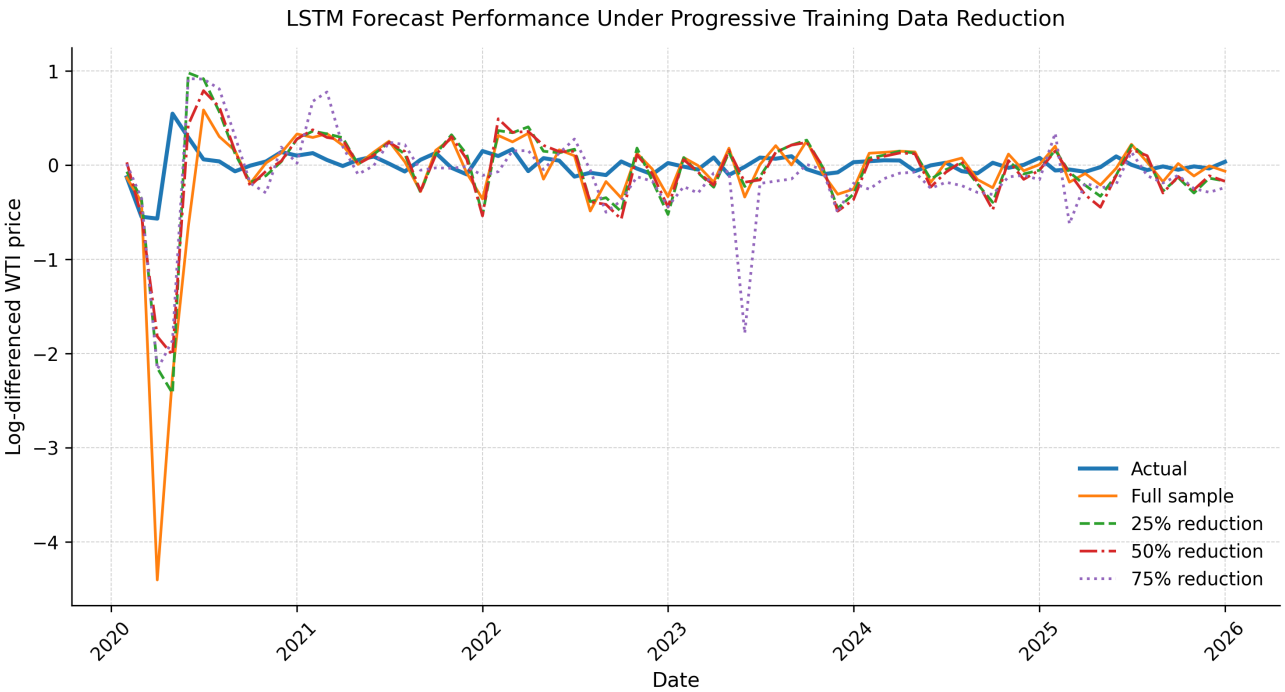

Figure 1: LSTM performance deteriorates as training data decreases, particularly under severe data reduction scenarios.

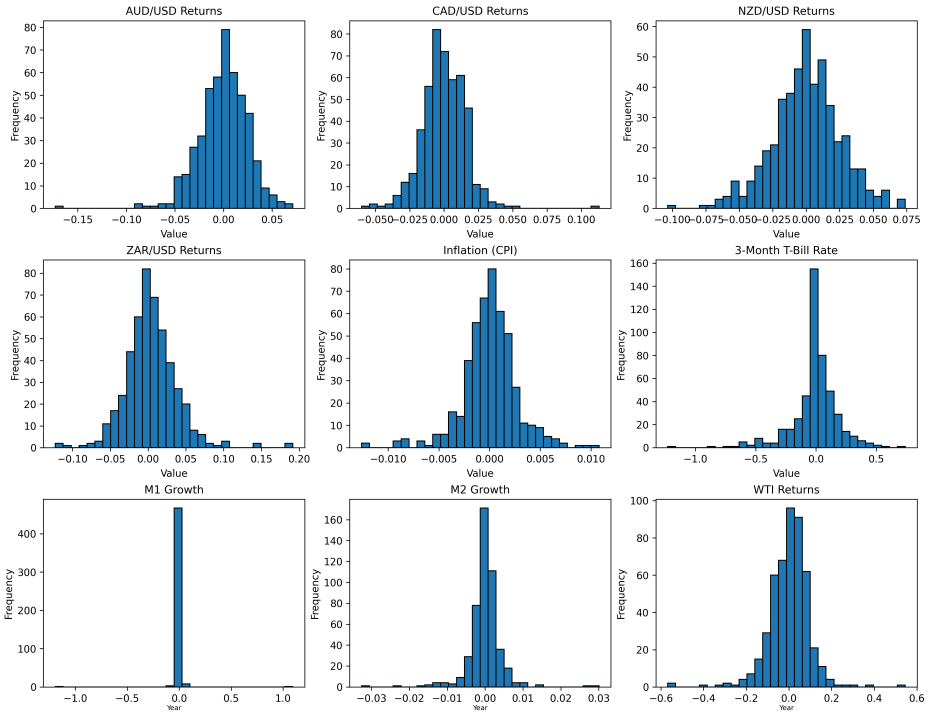

Figure 2: Distribution of oil price growth and predictor variables.

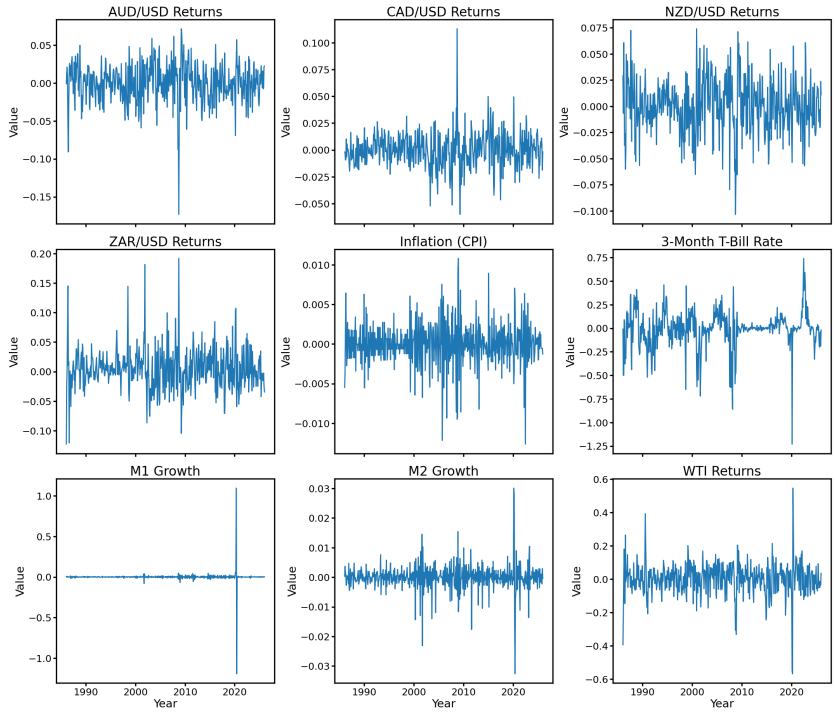

Figure 3: Time-series plots of oil price growth and predictor variables over the sample period..

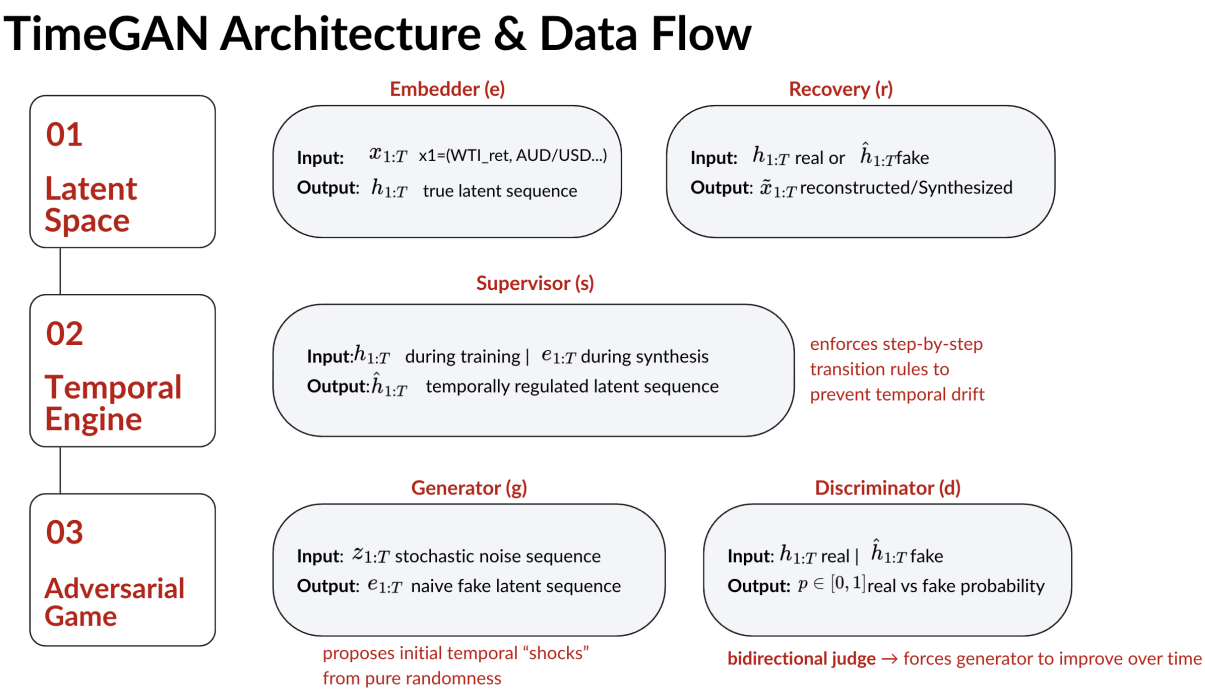

Figure 4: TimeGAN structure.

Loss functions. Each training stage optimises a distinct set of objectives. Let $\mathbf{x} \in \mathbb{R}^{T \times d}$ denote a real input sequence of length T with d features, $\mathbf{z} \in \mathbb{R}^{T \times d_z}$ a random noise sequence, $E(\cdot)$ the Embedder, $R(\cdot)$ the Recovery, $S(\cdot)$ the Supervisor, $G(\cdot)$ the Generator, and $D(\cdot)$ the Discriminator. We write $\mathbf{h} = E(\mathbf{x})$ for the real latent sequence, $\hat{\mathbf{h}} = G(\mathbf{z})$ for the synthetic latent sequence, and $\hat{\mathbf{x}} = R(\mathbf{h})$ for the reconstructed sequence.

Stage 1 — Autoencoder (Embedder + Recovery). The Embedder and Recovery are jointly trained

to minimise the sequence reconstruction error:

$$\mathcal{L}_{\text{AE}} = 10\,\sqrt{\mathcal{L}_{\text{recon}}}, \qquad \mathcal{L}_{\text{recon}} = \frac{1}{T}\sum_{t=1}^{T}\big\|\mathbf{x}_t - R(E(\mathbf{x}))_t\big\|_2^2 \tag{5}$$

The square-root scaling dampens large early reconstruction errors during initialisation, and the factor of 10 up-weights this objective relative to auxiliary terms used in later stages.

Stage 2 — Supervisor. The Supervisor is trained to predict the next latent state given the current one, supervised directly by the Embedder's representation of real sequences:

$$\mathcal{L}_S = \frac{1}{T-1}\sum_{t=1}^{T-1}\big\|\mathbf{h}_{t+1} - S(\mathbf{h})_t\big\|_2^2 \tag{6}$$

This one-step-ahead objective embeds the temporal transition dynamics of real data directly into the Supervisor and, through shared gradient paths, into the Generator.

Stage 3 — Joint training.

Embedder (joint refinement). During joint training the Embedder continues to be updated, balancing reconstruction fidelity with temporal coherence:

$$\mathcal{L}_E = 10\,\sqrt{\mathcal{L}_{\text{recon}}} \;+\; 0.1\,\mathcal{L}_S \tag{7}$$

Discriminator. The Discriminator is trained with binary cross-entropy on three terms: real sequences encoded into latent space, synthetic sequences passed through the Supervisor ($S(G(\mathbf{z}))$), and synthetic sequences from the Generator directly ($G(\mathbf{z})$), with the last term weighted by a hyperparameter γ:

$$\mathcal{L}_D = \underbrace{-\mathbb{E}\big[\log D(E(\mathbf{x}))\big]}_{\text{real}}\,\underbrace{-\mathbb{E}\big[\log(1 - D(S(G(\mathbf{z}))))\big]}_{\text{fake (supervised)}}\,\underbrace{-\gamma\,\mathbb{E}\big[\log(1 - D(G(\mathbf{z})))\big]}_{\text{fake (embedded)}} \tag{8}$$

The Discriminator is updated only when its loss exceeds a threshold of 0.15, preventing early discriminator dominance from collapsing Generator gradients.

Generator. The Generator loss combines four terms: two adversarial objectives (mirroring the two fake terms in $\mathcal{L}_D$), the supervised temporal loss, and a moment-matching loss on the first two moments of the generated output:

$$\mathcal{L}_G = \underbrace{\mathcal{L}_{\text{adv}}^{S} + \mathcal{L}_{\text{adv}}^{E}}_{\text{adversarial}} \;+\; 100\,\sqrt{\mathcal{L}_S} \;+\; 100\,\mathcal{L}_{\text{moment}} \tag{9}$$

where the moment-matching loss penalises differences in per-feature mean and standard deviation between real and synthetic outputs:

$$\mathcal{L}_{\text{moment}} = \sum_{j=1}^{d}\Big(\big|\mu_j^{\text{real}} - \mu_j^{\text{synth}}\big| \;+\; \big|\sigma_j^{\text{real}} - \sigma_j^{\text{synth}}\big|\Big) \tag{10}$$

The coefficients of 100 on the supervised and moment terms reflect the relative importance placed on temporal fidelity and first-moment accuracy versus the adversarial objective, preventing the Generator from collapsing to a distribution that fools the Discriminator but loses sequential structure.

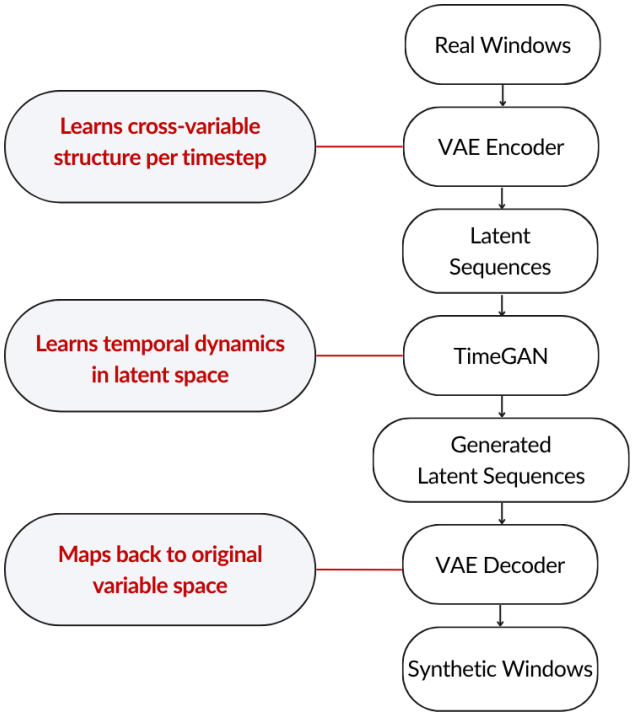

Figure 5: Proposed Architecture for VAE-TimeGAN.

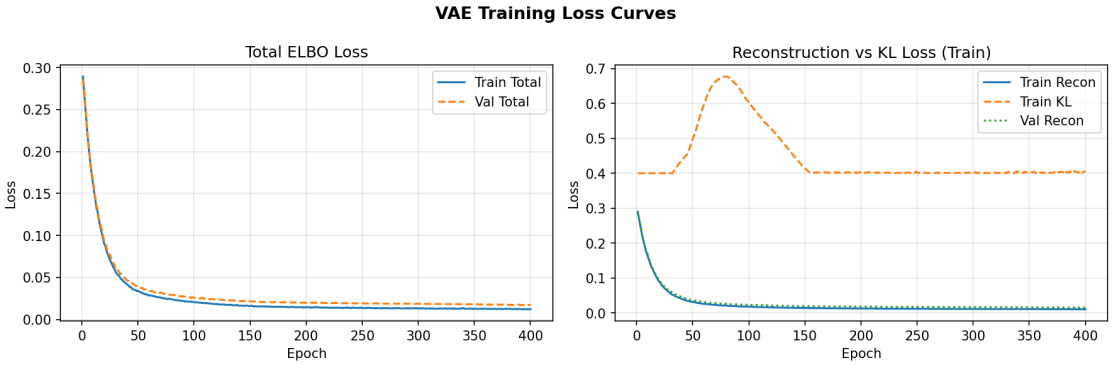

Figure 6: VAE Loss Curves.

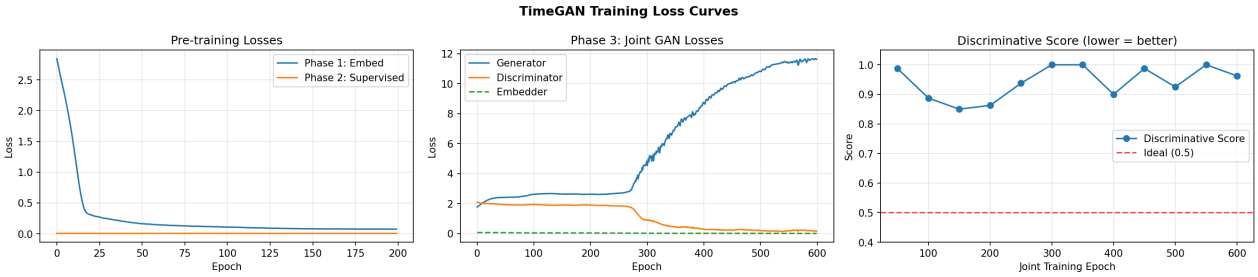

Figure 7: TimeGAN Loss Curves for VAE-TimeGAN training.

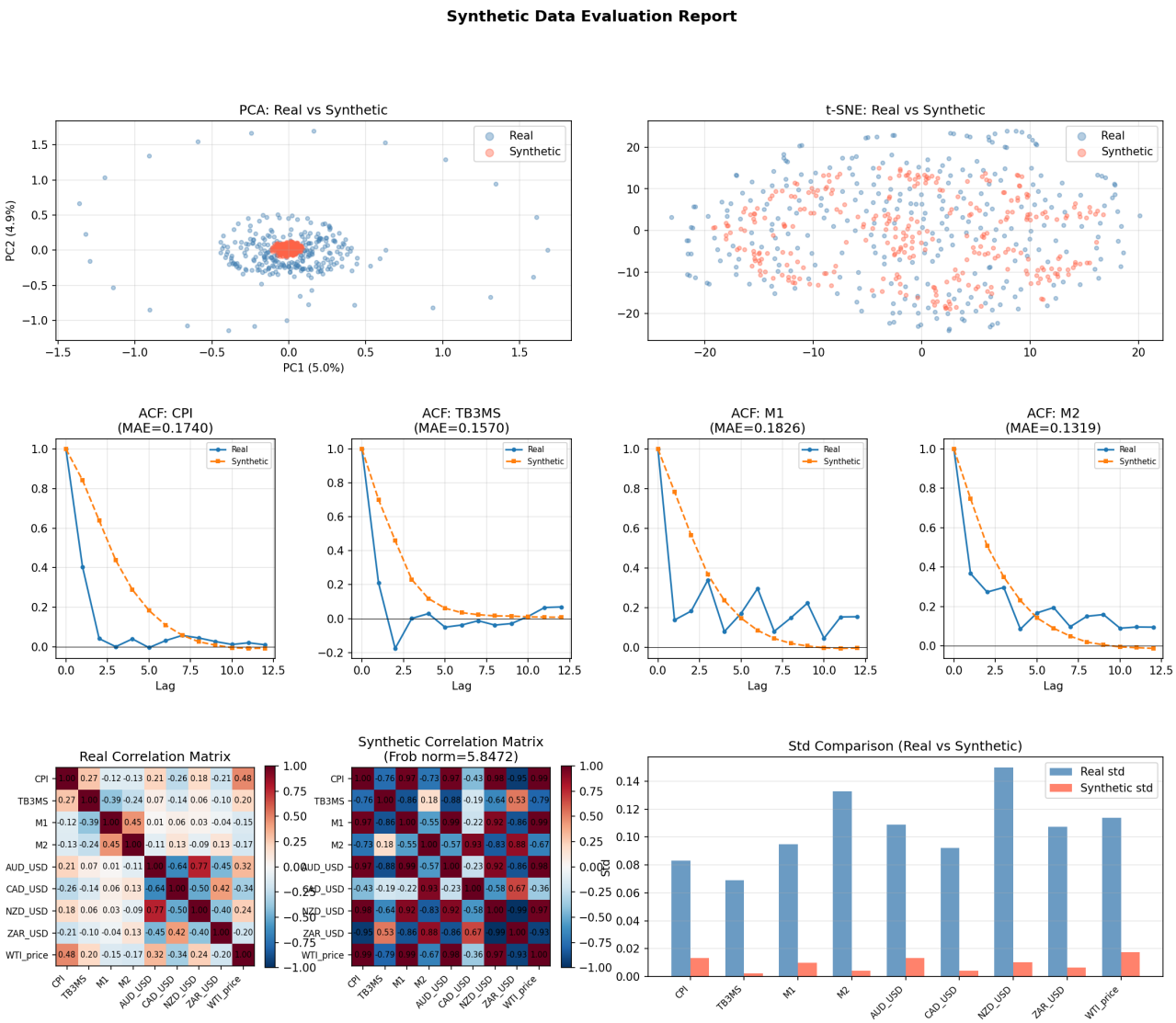

Figure 8: Evaluation results for VAE-TimeGAN training.

Table 1: VAE posterior collapse tuning progression.

Configuration	**KL at convergence**	**Latent SD**	**Posterior SD**	**Outcome**
8 latent dims, $\beta = 1.0$	0.0001	0.004–0.009	—	Full collapse
8 latent dims, $\beta = 0.01$	0.560	0.03–0.15	—	KL dominates
Free bits, $\beta_{\max} = 0.005$	0.800	0.05–0.18	—	Partial improvement
4 latent dims, free bits	0.400	0.15–0.28	0.38–0.94	Best result

Table 2: VAE hyperparameter configuration for VAE-TimeGAN.

Hyperparameter	**Value**
Latent Dimensions	4
Hidden dimension (all GRU layers)	64
Number of Epochs	400
Batch size	25
Learning rate	1×10^{-4}
KL Warmup Epochs (Number of Epochs Before KL Loss is added)	50
Maximum KL Weight	.005

Table 3: TimeGAN hyperparameter configuration for VAE-TimeGAN.

Hyperparameter	**Value**
Hidden Dimension	32
Number of Hidden Layers	2
Number of Epochs	400
Batch size	32
Learning rate	1×10^{-4}
Number of Epochs	1000
Frequency of Model Evaluation	50 Epochs

Table 4: VAE-TimeGAN evaluation results compared with standard TimeGAN.

Dimension	**Metric**	**VAE-TimeGAN**	**Std TimeGAN**	**Target**
Marginal distribution	Avg. KS statistic	0.5427	0.145	$\to 0$
	Avg. Wasserstein distance	—	0.020	$\to 0$
Joint structure	Correlation Frobenius norm	5.847	2.089	$\to 0$
Temporal dynamics	Avg. ACF MAE (all vars)	0.1769	0.203†	$\to 0$
Distributional	MMD	0.412	0.437	$\to 0$
Discriminability	Discriminative accuracy	1.000	—	$\to 0.5$

† Standard TimeGAN ACF computed as ACF L2 on WTI only; VAE-TimeGAN figure is mean ACF MAE across all nine variables, included for directional comparison only.
Standard TimeGAN evaluated on 62 out-of-sample windows (2019–2026); VAE-TimeGAN evaluated on 312 training windows. Direct numerical comparison should be interpreted with caution.

Table 5: Optimal LSTM Hyperparameters Across Feature Sets and Data Augmentation Methods

Feature Set	**Model**	**Lookback**	**Dropout**	**Units**	**Epochs**
WTI Only	LSTM (Baseline)	2	0.001	170	50
	LSTM (Jitter)	2	0.001	170	50
	LSTM (TimeGAN)	2	0.001	170	100
	LSTM (VAE-TimeGAN)	2	0.001	170	50
With TB3M	LSTM (Baseline)	2	0.001	170	50
	LSTM (Jitter)	2	0.001	170	50
	LSTM (TimeGAN)	2	0.1	50	50
	LSTM (VAE-TimeGAN)	2	0.1	170	50
8 Covariates	LSTM (Baseline)	2	0.001	50	50
	LSTM (Jitter)	10	0.1	170	50
	LSTM (TimeGAN)	2	0.1	50	50
	LSTM (VAE-TimeGAN)	2	0.1	170	50

Table 6: Econometric Model Specifications

Model	**Specification**
AR(3)	3 autoregressive lags
ADL(1,1)	1 lag of the dependent variable and 1 lag of each explanatory variable

Table 7: Model Performance (With COVID Period)

Model	**RMSE**	**MAE**	**Directional Accuracy (DA)**
WTI only			
AR(3)	0.136	0.0806	62.5%
TimeGAN	0.551	0.1917	55.6%
Baseline	0.605	0.2667	59.7%
Jitter	0.608	0.2501	58.3%
VAE-TimeGAN	0.640	0.2565	56.9%
TB3M			
ADL(1,1)	0.137	0.0830	58.3%
Baseline	0.436	0.2138	58.3%
Jitter	0.467	0.2226	58.3%
TimeGAN	0.556	0.2681	63.9%
VAE-TimeGAN	0.667	0.2859	58.3%
All covariates			
Full ADL	0.152	0.0910	51.4%
Baseline	0.402	0.2810	63.9%
TimeGAN	0.760	0.3948	58.3%
Jitter	0.776	0.5808	45.8%
VAE-TimeGAN	0.783	0.5634	52.8%

Table 8: Model Performance (No COVID Period)

Model	**RMSE**	**MAE**	**Directional Accuracy (DA)**
WTI only (excluding COVID crash)			
AR(3)	0.070	0.0591	61.2%
TimeGAN	0.123	0.1013	53.7%
Jitter	0.189	0.1552	58.2%
VAE-TimeGAN	0.192	0.1568	56.7%
Baseline	0.210	0.1712	59.7%
TB3M (excluding COVID crash)			

Model	RMSE	MAE	Directional Accuracy (DA)
ADL(1,1)	0.072	0.0603	56.7%
Baseline	0.189	0.1473	56.7%
Jitter	0.191	0.1494	56.7%
VAE-TimeGAN	0.257	0.1892	56.7%
TimeGAN	0.345	0.2054	62.7%
All covariates (excluding COVID crash)			
Full ADL	0.077	0.0635	52.2%
Baseline	0.334	0.2607	62.7%
TimeGAN	0.390	0.2967	56.7%
Jitter	0.673	0.5316	44.8%
VAE-TimeGAN	0.694	0.5233	50.7%

Table 9: Trading Performance (With COVID Period)

Strategy	Ann. Sharpe	Ann. Return (%)	Cum. Return (%)	Max Drawdown (%)
Buy and Hold	0.015	-12.45	-54.97	80.49
WTI only				
TimeGAN	0.522	13.81	117.35	54.61
Jitter	0.326	4.41	29.36	67.91
VAE-TimeGAN	0.304	3.43	22.43	67.91
Baseline	0.237	0.40	2.40	67.91
TB3M				
Jitter	0.597	17.78	166.89	54.56
Baseline	0.557	15.78	140.85	54.56
TimeGAN	0.310	4.00	26.53	67.38
VAE-TimeGAN	0.226	0.34	2.08	68.71
All Covariates				
Baseline	1.063	45.59	852.30	17.97
VAE-TimeGAN	0.478	11.87	96.01	54.56
TimeGAN	0.477	11.85	95.82	54.56
Jitter	-0.074	-14.25	-60.23	67.92

Table 10: Trading Performance (No COVID Period)

Strategy	**Ann. Sharpe**	**Ann. Return (%)**	**Cum. Return (%)**	**Max Drawdown (%)**
Buy and Hold	0.325	5.18	32.59	-53.54
WTI only (excluding COVID crash)				
TimeGAN	0.671	10.75	76.83	22.90
Jitter	0.596	9.37	64.88	22.49
VAE-TimeGAN	0.473	6.95	45.49	22.49
Baseline	0.419	5.88	37.57	23.93
TB3M (excluding COVID crash)				
Jitter	0.816	13.75	105.30	17.53
Baseline	0.646	10.09	71.00	17.53
VAE-TimeGAN	-0.123	-3.45	-17.82	52.86
TimeGAN	-0.162	-3.93	-20.08	48.16
All Covariates (excluding COVID crash)				
Baseline	0.715	11.43	83.02	17.97
VAE-TimeGAN	0.434	6.12	39.36	27.65
TimeGAN	0.354	4.90	30.63	38.51
Jitter	0.137	0.89	5.04	39.07

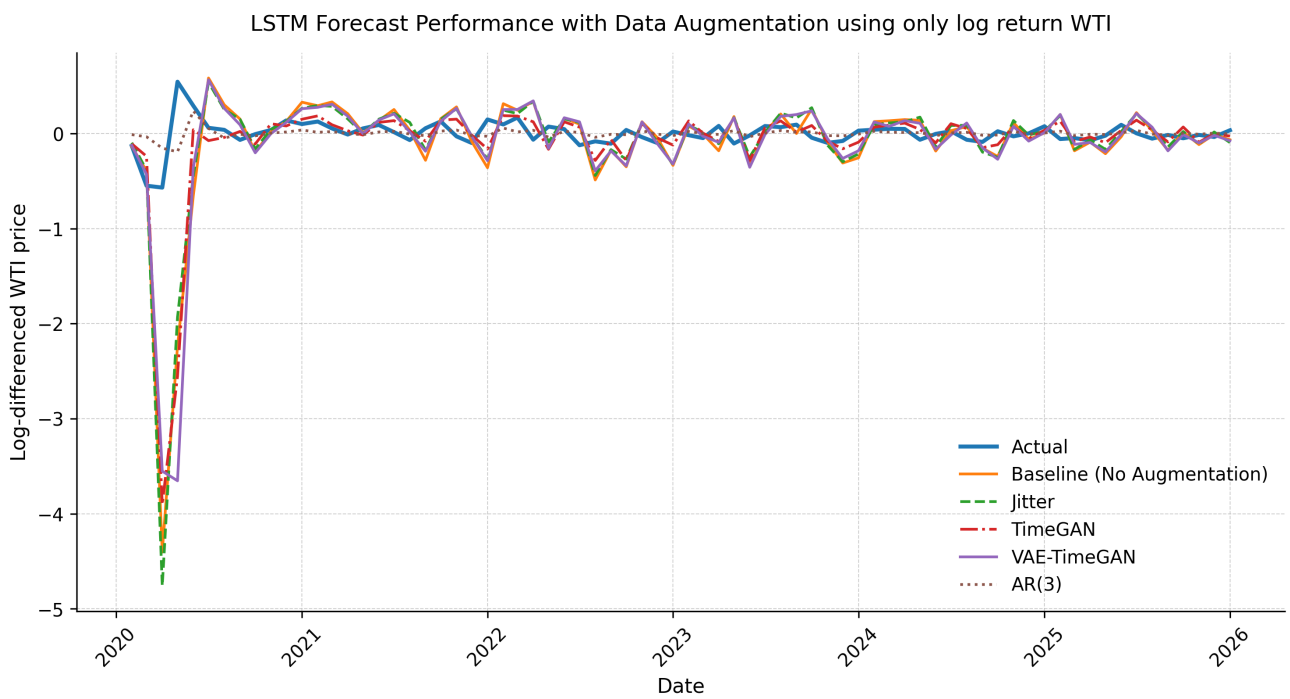

Figure 9: Out-of-sample comparison of actual and predicted values using only WTI returns.

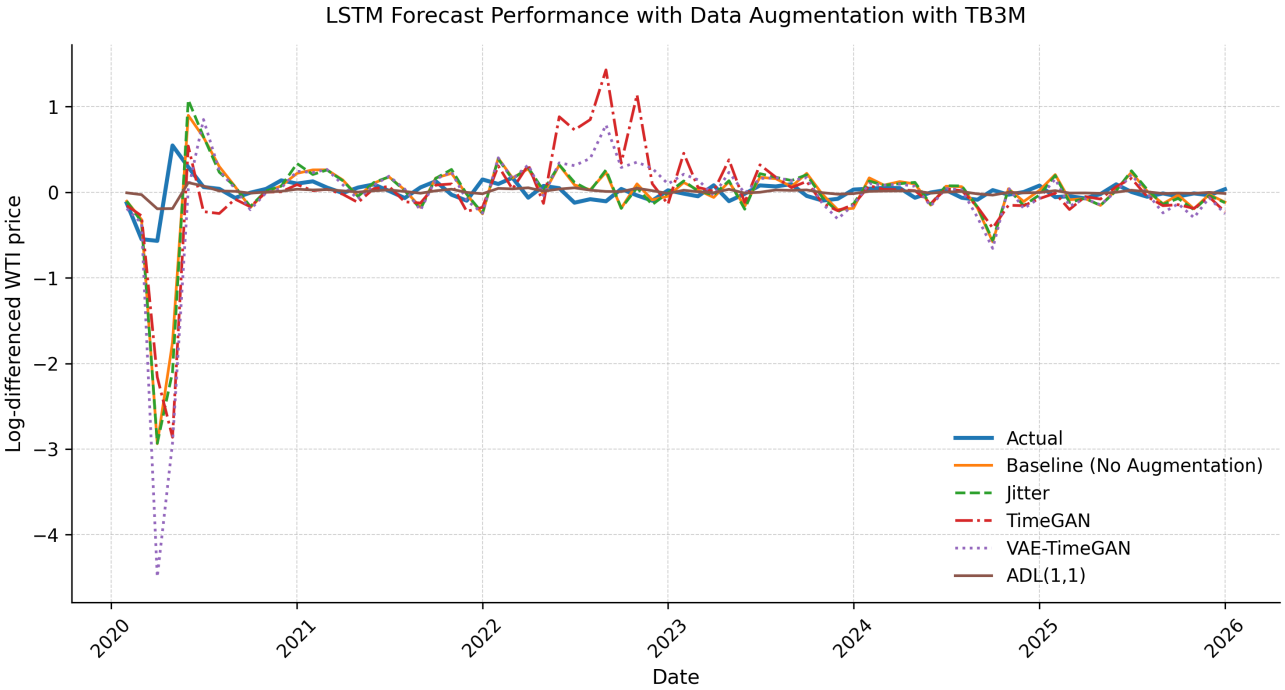

Figure 10: Out-of-sample comparison of actual and predicted values using WTI returns and the 3-month Treasury Bill rate (TB3M).

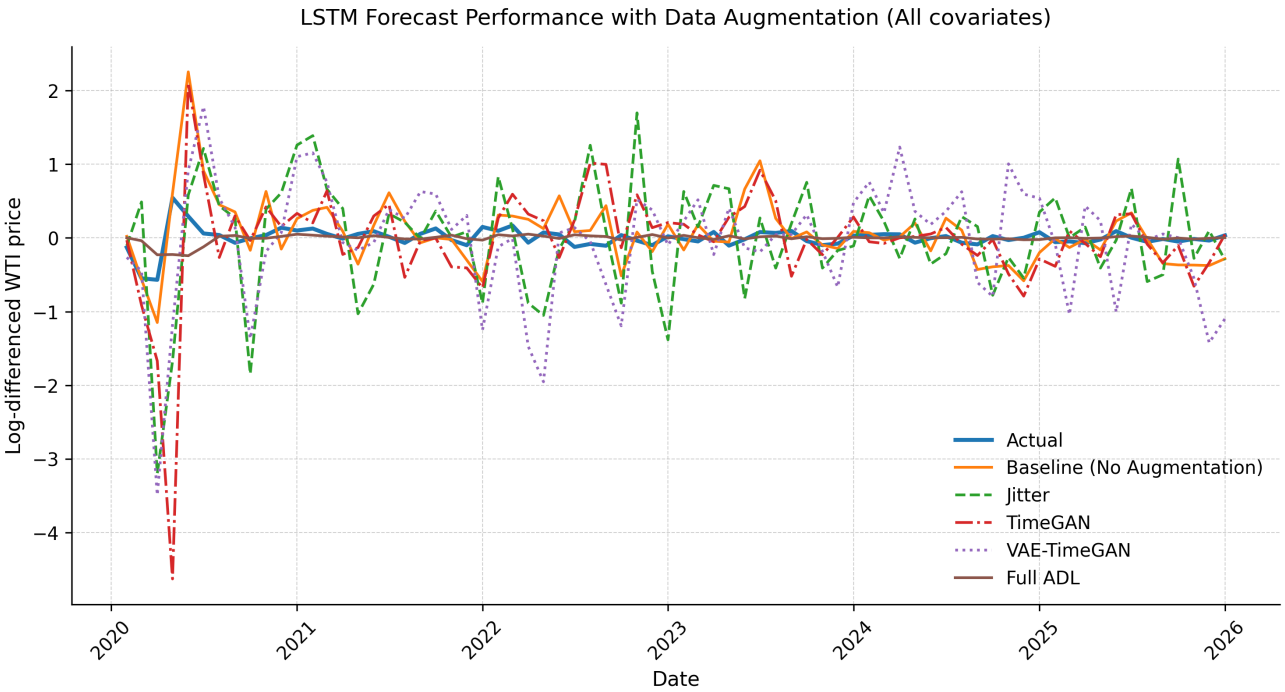

Figure 11: Out-of-sample comparison of actual and predicted values using WTI returns and the full set of covariates.

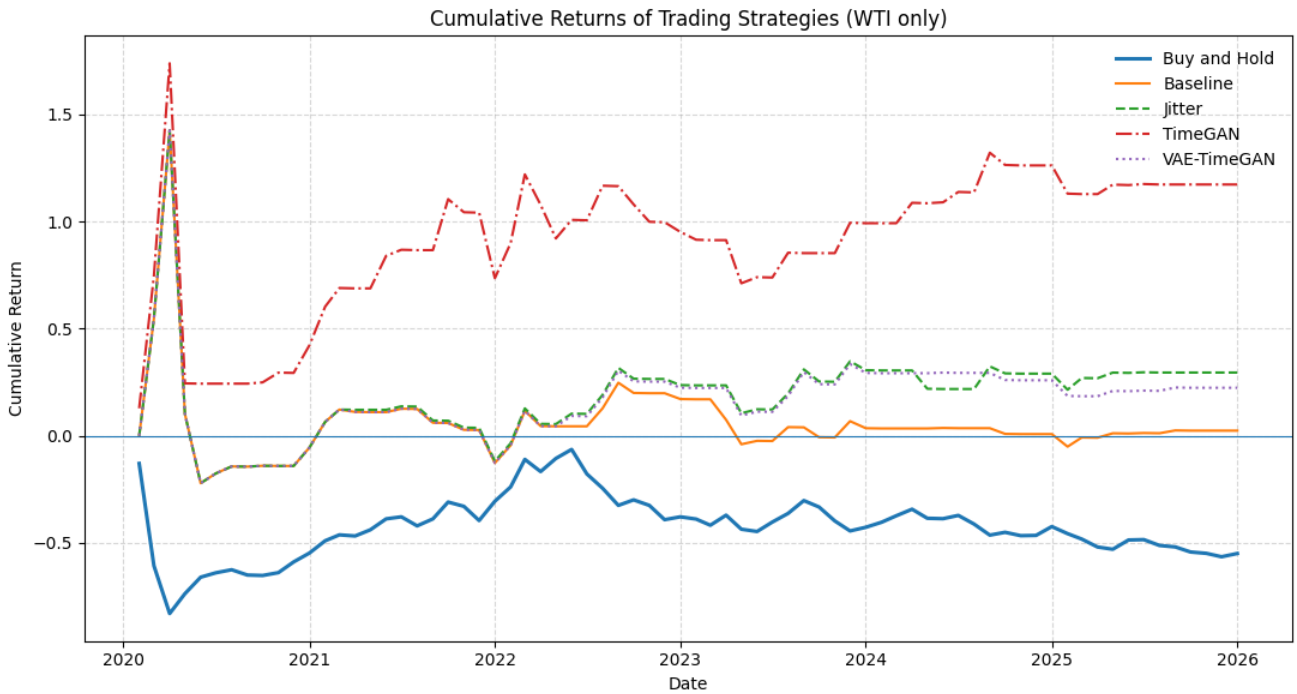

Figure 12: Cumulative Returns of Trading Strategies (WTI only)

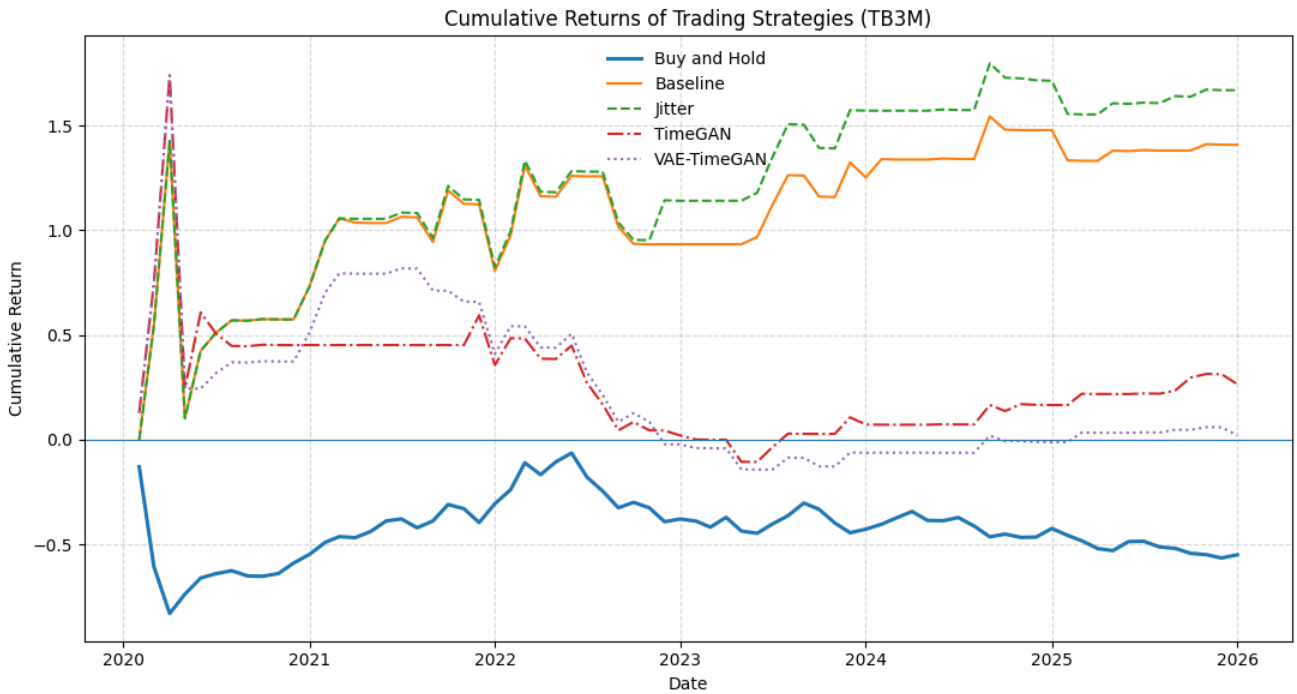

Figure 13: Cumulative Returns of Trading Strategies (WTI + TB3M)

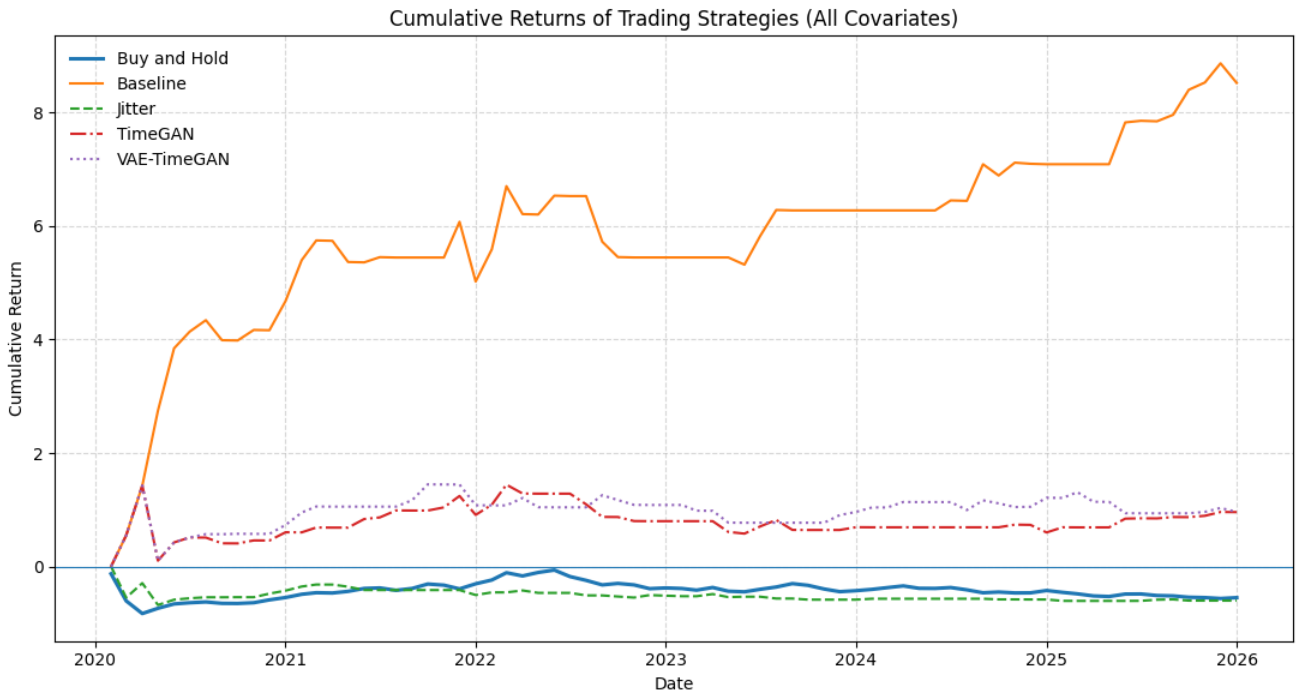

Figure 14: Cumulative Returns of Trading Strategies (All Covariates)

References

Abbasi, M., Kargar, M., Ahmadian, F., NoormohammadZadehMaleki, D., Arandan, A., and Hosseini, N. S. (2024). Gn-cnn-lstm: Financial market prediction with gaussian noise embedded cnn lstm. In *2024 11th International Symposium on Telecommunications (IST)*, pages 287–294. IEEE.

Arora, V. and Tanner, M. (2012). How important are real interest rates for oil prices? *MPRA Paper No. 35883, Munich Personal RePEc Archive*.

Conlon, T., Cotter, J., and Eyiah-Donkor, E. (2024). Forecasting the price of oil: A cautionary note. *Journal of Commodity Markets*, 33:100378.

Giantsidi, S. and Tarantola, C. (2025). Deep learning for financial forecasting: A review of recent trends. *International Review of Economics & Finance*, page 104719.

Iwana, B. K. and Uchida, S. (2021). An empirical survey of data augmentation for time series classification with neural networks. *Plos one*, 16(7):e0254841.

Kingma, D. P. and Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

Ly, R., Traore, F., and Dia, K. (2021). Forecasting commodity prices using long-short-term memory neural networks. *arXiv preprint arXiv:2101.03087*.

Ng, S. and McCracken, M. W. (2015). Fred-md: A monthly database for macroeconomic research. Working Paper 2015-012, Federal Reserve Bank of St. Louis.

Wen, Q., Sun, L., Yang, F., Song, X., Gao, J., Wang, X., and Xu, H. (2020). Time series data augmentation for deep learning: A survey. *arXiv preprint arXiv:2002.12478*.

Yin, X., Han, Y., Xu, Z., and Liu, J. (2021). Vaecgan: a generating framework for long-term prediction in multivariate time series. *Cybersecurity*, 4(1):22.

Yoon, J., Jarrett, D., and van der Schaar, M. (2019). Time-series generative adversarial networks. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.